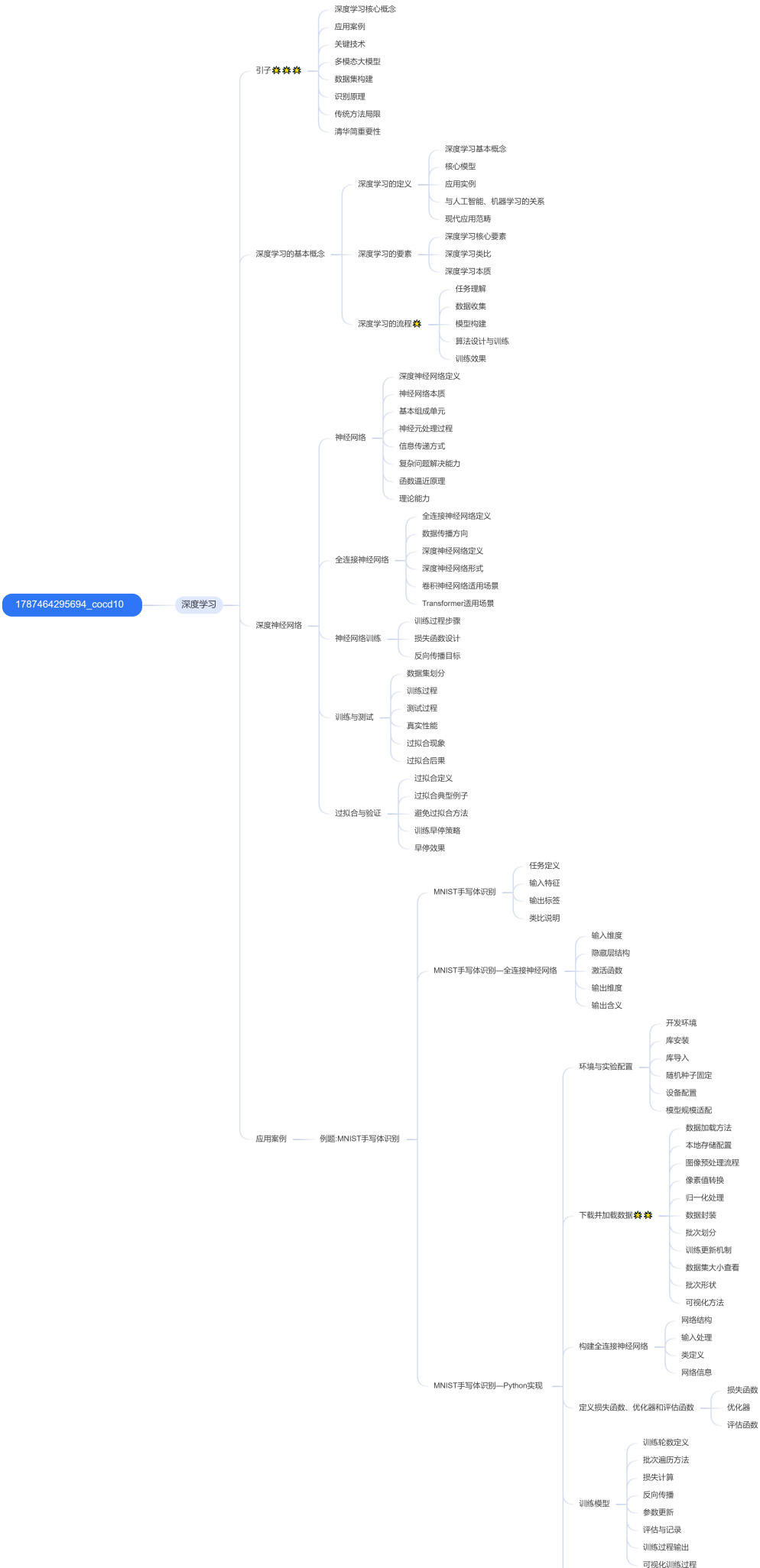


以下为AI生成的大纲笔记的内容

---



## 一、深度学习 00:18

### 1.引子 🌟🌟🌟 00:26

- 深度学习核心概念：让计算机通过大量数据自动学习规律，无需人工逐字破译。
- 应用案例：利用深度学习识别清华简中的战国楚文字，提升识读效率十倍以上。
- 关键技术：卷积神经网络（CNN）提取字形特征，Transformer模型捕捉复杂结构信息。
- 多模态大模型：同时理解图像中的文字及其含义，增强识别能力。
- 数据集构建：以清华简为原型，创建包含上千种文字、近两万张图像的数据集。
- 识别原理：训练后的模型自动提取陌生字形的笔画结构特征，与数据库已知文字比对匹配。
- 传统方法局限：专家需一字一字破译，数万个古文字耗时巨大，效率低。
- 清华简重要性：填补考古学、语言学、思想史、音乐史等领域空白，包含失传文献和最早十进制计算器。

### 2.深度学习的基本概念 02:58

#### 1) 深度学习的定义 03:15

- 深度学习基本概念：利用特定模型（神经网络）挖掘数据中隐藏的模式及输入输出关联的技术。
- 核心模型：深度神经网络（DNN）。
- 应用实例：MNIST手写体识别。
- 与人工智能、机器学习的关系：人工智能模拟人类行为，机器学习在其范畴内模拟从数据中学习规律，深度学习进一步利用神经网络挖掘数据。
- 现代应用范畴：生成式人工智能技术，如ChatGPT、DeepSeek等大语言模型，以及视频生成模型（如C Dances or A），均属于深度学习。

#### 2) 深度学习的要素 03:53

- 深度学习核心要素：模型和数据。
- 深度学习类比：火箭发射，引擎为神经网络模型，燃料为海量数据。
- 深度学习本质：在大数据中训练神经网络模型的过程。

#### 3) 深度学习的流程 🌟 04:19

- 任务理解：明确处理任务类型，如识别图像、语音、视频中的对象与结构，或处理文本、语音等序列数据，或生成图像视频等新内容。
- 数据收集：根据任务收集相应数据，如借助图像处理技术分割字符区域，并借助专家标注形成一一对应的数据集。
- 模型构建：根据任务输入输出特点，选择适配的深度神经网络，如全连接神经网络或卷积神经网络。
- 算法设计与训练：设计适合的算法，利用反向传播算法得到优化方向，构建优化方法更新网络参数，实现模型训练。
- 训练效果：训练使神经网络具备解决任务的能力，如使模型从无法识别古文字到具备正确识别能力。

### 3.深度神经网络 06:09

#### 1) 神经网络 06:14

- 深度神经网络定义：深度学习核心，底层构架为神经网络（neural network）。
- 神经网络本质：模仿人类大脑结构与功能的计算模型。
- 基本组成单元：大量神经元，单个神经元结构包括线性组合与非线性变换。
- 神经元处理过程：输入信息先进行线性组合，再通过非线性激活函数进行非线性变换。
- 信息传递方式：输出输入到下一层神经元。
- 复杂问题解决能力：简单结构经相互连接可解决复杂问题。

- 函数逼近原理：只要神经元足够多，单层神经元也可逼近任意输入输出间的函数关系。
- 理论能力：神经网络从理论上可学习任何任务，如古文字字符识别。

## 2) 全连接神经网络 07:02

- 全连接神经网络定义：最简单的神经网络，具有多层神经元结构。
- 数据传播方向：数据沿着输入层通过一层或多层隐藏层，单向传播到输出层。
- 深度神经网络定义：含两个隐藏层以上的神经网络。
- 深度神经网络形式：多种多样，如卷积神经网络、Transformer、生成对抗网络、扩散模型等。
- 卷积神经网络适用场景：适合建模图像中像素之间的邻近关系。
- Transformer适用场景：适合建模文本中词语的语义交互。

## 3) 神经网络训练 07:37

- 训练过程步骤：输入样本→前向计算→得到预测值→计算误差→反向传播更新参数。
- 损失函数设计：根据不同任务设计不同损失函数，如分类任务用交叉熵，回归任务用方差。
- 反向传播目标：利用反向传播算法更新网络参数，不断降低损失函数，使预测值接近真实值。

## 4) 训练与测试 08:11

- 数据集划分：将数据集分为训练数据集和测试数据集。
- 训练过程：在训练数据集上训练深度神经网络。
- 测试过程：将训练好的模型在测试数据集上测试，评估性能（如分类准确率）。
- 真实性能：测试结果代表模型在任务中的真实性能。
- 过拟合现象：模型在训练中不断拟合训练数据，导致学到错误的输入输出关系。
- 过拟合后果：模型无法在未知数据中表现良好。

## 5) 过拟合与验证 08:43

- 过拟合定义：模型在训练数据上表现优异，但在未知数据上表现差的现象。
- 过拟合典型例子：一朝被蛇咬，十年怕井绳，错误认为所有类似形状物体都会咬人。
- 避免过拟合方法：从训练数据中拆分出验证集，模拟未知数据。
- 训练早停策略：在训练过程中，当验证集损失函数不再降低时停止训练。
- 早停效果：使模型在未知数据集中具有更好的性能。

## 4.应用案例 09:21

### 1) 例题:MNIST手写体识别

- MNIST手写体识别 09:37
  - 任务定义：MNIST手写体识别是识别图像中0-9数字的十类图像分类任务。
  - 输入特征：输入图像为 $28 \times 28$ 像素，每个像素值为0-255的整数，代表灰度值。
  - 输出标签：输出标签为图像对应的数字0-9。
  - 类比说明：与古文字识别类似，均属于图像识别问题。
- MNIST手写体识别—全连接神经网络 10:05
  - 输入维度：784维原始图像。
  - 隐藏层结构：两层128维隐藏层。
  - 激活函数：采用ReLU作为非线性激活函数。
  - 输出维度：10维神经元。
  - 输出含义：预测图像属于0到9数字的概率分布。
- MNIST手写体识别—Python实现 10:25
  - 环境与实验配置 10:48
    - 开发环境：使用Visual Studio Code作为Python集成开发环境。
    - 库安装：执行特定代码行安装所需的Python库。
    - 库导入：使用import命令导入已安装的Python库。

- 随机种子固定：固定随机种子以确保结果可复现。
- 设备配置：根据实验环境灵活设置训练推理在CPU或GPU上进行。
- 模型规模适配：由于模型规模较小，可在CPU上进行训练推理。
- 下载并加载数据 11:21
  - 数据加载方法：直接调用`datas`中的`MNIST`类下载MNIST数据集。
  - 本地存储配置：通过设置`root`参数，配置数据下载后的本地存储地址。
  - 图像预处理流程：使用`transforms.Compose`对输入图像进行预处理。
  - 像素值转换：将像素从0-255的整数转换为0-1之间的实数。
  - 归一化处理：归一化到均值0.5、方差0.5。
  - 数据封装：将预处理数据转化为预定义的`Data`类。
  - 批次划分：使用`DataLoader`将数据分成不同批次，批次大小设置为64。
  - 训练更新机制：模型训练时，每一步计算一个批次64个样本的损失函数平均值，以此更新权重。
  - 数据集大小查看：通过`DataLoader`的`len`函数查看训练集和测试集大小，训练集有60000张图片，测试集有10000张图片。
  - 批次形状：一个批次大小为64，图像为单通道，大小为 $28 \times 28$ 。
  - 可视化方法：通过`imshow`函数绘制灰度图片，并与对应标签一同可视化，可查看第一个批次的前12个数据。
- 构建全连接神经网络 13:17
  - 网络结构：包含两个大小为128的隐藏层，使用ReLU非线性激活函数。
  - 输入处理：全连接层输入为一维向量，需将 $28 \times 28$ 图片展平为784维向量。
  - 类定义：定义一个名为`AMistClassifier`的类来描述网络结构。
  - 网络信息：通过`print`函数输出网络结构及可训练参数总量。
- 定义损失函数、优化器和评估函数 13:55
  - 损失函数：交叉熵，多分类任务常用损失函数。
  - 优化器：Adam，基于反向传播算法，根据梯度自适应调整学习率。
  - 评估函数：通过for循环计算模型在给定数据集上的平均损失函数值及手写体识别准确率。
- 训练模型 14:25
  - 训练轮数定义：设置`epochs`为5，即遍历整个训练集五遍。
  - 批次遍历方法：通过for循环依次遍历`train_loader`中的所有批次。
  - 损失计算：在每个批次中计算损失函数。
  - 反向传播：通过`loss.backward()`执行反向传播算法，计算梯度。
  - 参数更新：使用`optimizer.step()`基于Adam优化器更新网络参数。
  - 评估与记录：每遍历一遍数据，利用评估函数计算模型在训练集和测试集中的损失及准确率，并存储在`history`变量中。
  - 训练过程输出：每个批次训练结束后，通过`print`函数输出训练损失、训练准确率、测试损失和测试准确率。
  - 可视化训练过程：训练结束后，利用`matplotlib`绘制`history`中存储的损失和准确率曲线。
- 测试集评估与混淆矩阵 16:09
  - 测试集评估：在测试集上测试训练好的模型，评估性能。
  - 评估指标：除损失和识别准确率外，可绘制混淆矩阵。
  - 混淆矩阵定义：行表示真实类别，列表示预测类别。
  - 主对角线意义：数值越大，代表该类别识别越准确。
  - 错误模式分析：可发现典型识别错误，如五被误识别为六，因形态接近。
- 查看若干预测结果 16:59
  - 预测结果可视化：将模型在测试集中的预测结果进行可视化展示。

- 查看批次样本：通过代码查看测试集中第一个批次的前12个例子的识别结果。
- 对比真实标签：将预测结果与真实标签进行对比，判断识别准确性。
- 错误预测示例：仅有一个样本被预测错误，将数字5错误预测为6。
- 全连接神经网络应用：以上步骤展示了在Python中构建全连接神经网络实现MNIST手写体识别的训练测试过程。

以下为AI生成的图文笔记的内容